

UART Communication and Data Integrity

Module	ET4047
Date	04/10/2026
Lab No.	Lab 5
Author	Luke Griffin

1 Setup

Equipment

Item	Qty	Notes
Arduino Nano (classic) or Nano Every, on Uno-style breakout shield	2	All the code works on either board. Task 1 detects the board automatically.
16×2 LCD with I2C backpack, or Adafruit RGB LCD shield	1-2	Either works: there is a separate set of sketches for each display (see below).
Jumper wires (male-male / male-female)	~10	
USB cables	2	One per board.
Push button	1	Task 3 only.

Software

- Arduino IDE. Select the board you have: *Tools* → *Board* → *Arduino AVR Boards* → *Arduino Nano* (classic), or *Arduino megaAVR Boards* → *Arduino Nano Every* (install the megaAVR package from the Boards Manager first). Classic Nano clones may also need *Tools* → *Processor* → *ATmega328P (Old Bootloader)*.
- **The library for your display only.** LiquidCrystal I2C (by Frank de Brabander) for the I2C backpack LCD, or Adafruit RGB LCD Shield Library (by Adafruit) for the RGB shield. *Tools* → *Manage Libraries...* and search for it.
- `SoftwareSerial` is built in. There is nothing to install.
- **The lab code**, `W05_UART_Lab_Code.zip`. Unzip it somewhere you can find it again (see the next section).

Choose the code for your display

There are two kinds of LCD in the lab, and the zip file has a folder of sketches for each one. Work out which display you have and use **only** that folder. The sketches in the two folders do exactly the same thing; only the lines that set up the LCD are different.

Display	How to recognise it	Folder to use	Library
16×2 LCD with I2C backpack	A separate screen with a small board on the back and four wires: GND, VCC, SDA, SCL	<code>I2C_Backpack_LCD</code>	LiquidCrystal I2C
Adafruit RGB LCD shield	Plugs straight onto the breakout board and has five push buttons	<code>Adafruit_RGB_LCD_Shield</code>	Adafruit RGB LCD Shield Library

Each folder has one sub-folder per sketch (Task1_Loopback, Task2_Sender, Task2_Receiver, Task3_Sender, Task3_Receiver, Extension_Sender_ACK and Extension_Receiver_ACK), so each one opens directly in the Arduino IDE with File → Open. Every sketch is a **single .ino file**: there are no extra tabs or .h files to add. The code is not printed in this document; open each sketch and read its comments as you work through the task.

What is different between the two versions?

Only the LCD set-up at the top of the sketch. The **I2C backpack** version includes `LiquidCrystal_I2C.h`, creates `LiquidCrystal_I2C lcd(0x20, 16, 2)` (I2C address, columns, rows) and starts it with `lcd.init()` and `lcd.backlight()`.

The **RGB shield** version includes `Adafruit_RGBLCDShield.h`, creates `Adafruit_RGBLCDShield lcd` (the shield is always at address 0x20) and starts it with `lcd.begin(16, 2)` and `lcd.setBacklight(WHITE)`.

After that both use the same calls: `lcd.print()`, `lcd.write()`, `lcd.setCursor(col, row)` and `lcd.clear()`.

LCD wiring (all tasks)

I2C LCD connections

From		To	Purpose
LCD GND	→	Nano GND	Ground
LCD VCC	→	Nano 5V	Power
LCD SDA	→	Nano A4	I2C data
LCD SCL	→	Nano A5	I2C clock

LCD wiring (every task)

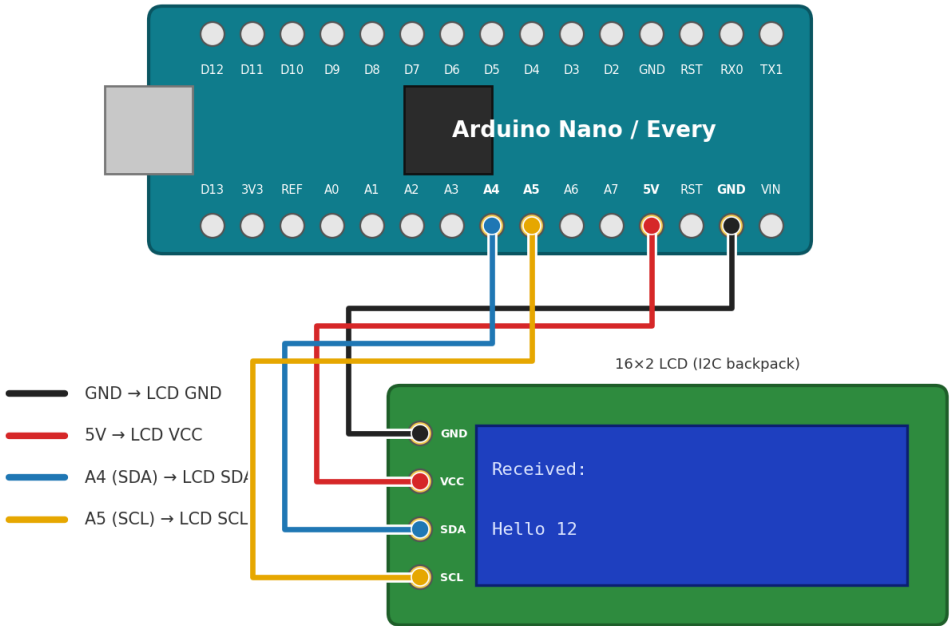


Figure 1: LCD wiring. The pinout is the same on the classic Nano and the Nano Every.

Using the Adafruit RGB LCD shield instead? Plug it onto the Uno-style breakout shield. It uses the same I2C pins (SDA on A4, SCL on A5), so no extra wiring is needed. Use the sketches in the [Adafruit_RGB_LCD_Shield](#) folder.

2 Task 1: UART loopback on a single board

Objective: connect the board’s UART output straight back into its own input. Everything the board transmits should arrive back on its receive pin, which proves that the UART, the wiring and your code are working before you involve a second board.

Wiring

Loopback connection (plus the LCD wiring from Section 1)

From		To	Purpose
Nano D1 (TX)	→	Nano D0 (RX)	Loopback: output feeds straight into input

Task 1: UART loopback

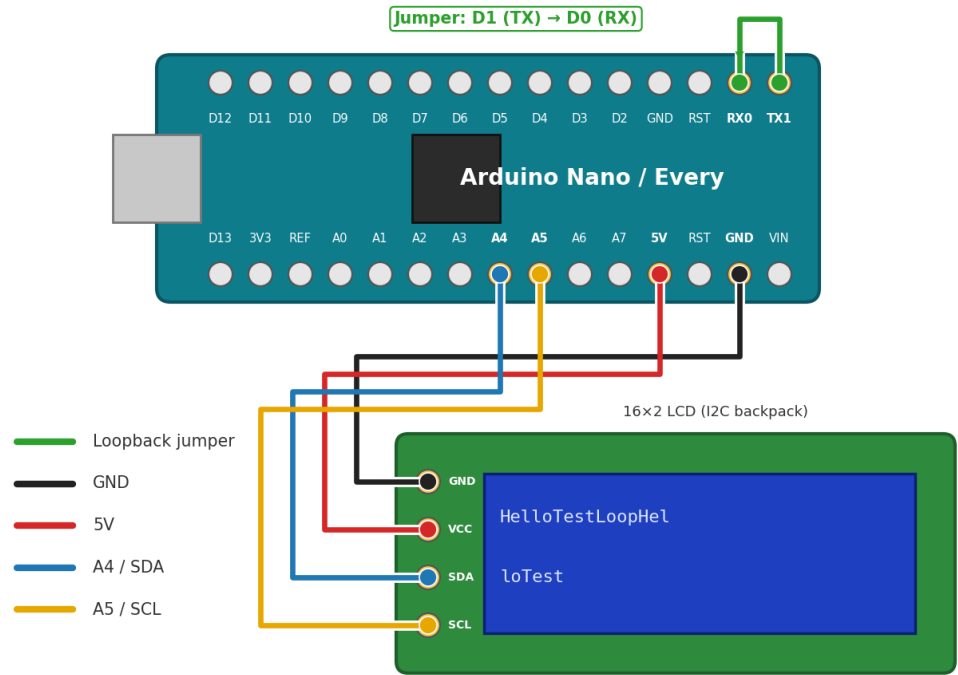


Figure 2: Task 1 wiring: one jumper from D1 (TX) to D0 (RX), which are next to each other on the header

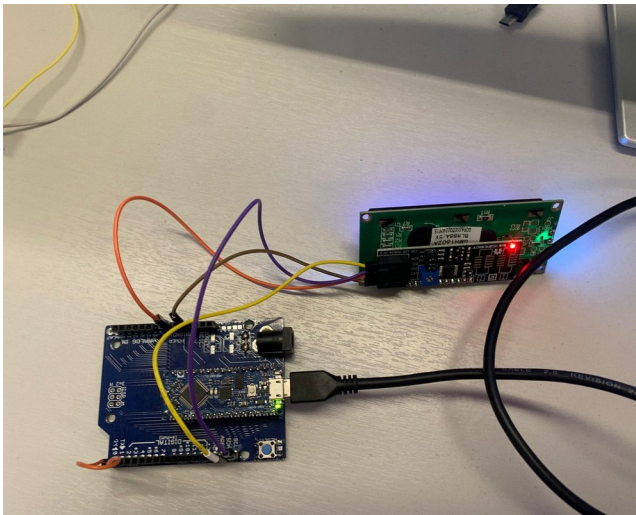


Figure 3: Loopback jumper and LCD wiring



Figure 4: LCD without line wrapping

	Classic Nano (ATmega328P)	Nano Every (ATmega4809)
Hardware UARTs on the pins	One, <code>Serial</code> . It is shared with the USB link.	A spare one, <code>Serial1</code> . USB has its own channel.
Port used by the Task 1 code	<code>Serial</code>	<code>Serial1</code>
Loopback jumper while uploading	Remove it , or the upload fails	Can stay in place

	Classic Nano (ATmega328P)	Nano Every (ATmega4809)
Serial Monitor during Task 1	Shows the messages being sent. Don't type in it: typed characters also arrive on D0.	Stays empty

Why the classic Nano behaves differently

On the classic Nano, D0 and D1 are also wired to the USB-to-serial chip. Anything the board sends on D1 goes both to the jumper and to your PC, and the upload uses the same pins, so the jumper gets in the way while uploading.

The code

Sketch: Task1_Loopback.ino

Open it from the folder for **your** display:

I2C backpack LCD: `I2C_Backpack_LCD/Task1_Loopback`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Task1_Loopback`

The two versions are the same except for the lines that set up the LCD. Read the comments in the sketch as you go: they explain every step.

How it works

- **Choosing the port.** The `#if` block runs at compile time. The Nano Every's board package defines `ARDUINO_AVR_NANO_EVERY`, so `LINK` becomes `Serial1`; on a classic Nano it becomes `Serial`. After that, every `LINK` in the code uses the right port, with no other changes needed.
- **The LCD object** is the only part that differs between the two versions (see *Choose the code for your display*). If your screen stays blank, check the SDA and SCL wiring and that the right library is installed. On the I2C backpack, also check the address in `LiquidCrystal_I2C lcd(0x20, 16, 2)`: 0x27 and 0x3F are also common.
- `const char* const messages[]` is an array of strings; both the pointers and the text are read-only. `msgIndex` steps through the messages, and an `if` sets it back to 0 after the last one.
- `LINK.begin(9600)` starts the hardware UART at 9600 baud. Anything printed to `LINK` leaves on D1 and, through the jumper, arrives on D0.
- **The millis() timer.** `millis()` returns milliseconds since power-up. Checking `millis() - lastSend >= SEND_INTERVAL` sends once a second without `delay()`, so `loop()` keeps running and never misses incoming bytes. `lastSend += SEND_INTERVAL` keeps the period exact, and the subtraction still works when `millis()` wraps round after about 49 days.
- **The receive loop.** `LINK.available()` returns how many bytes are waiting in the receive buffer; `LINK.read()` takes one out. Line endings are skipped and every other byte goes to `lcdPutChar()`.
- `lcdPutChar()` **wraps the text.** It remembers the cursor position in `col` and `row`. When a row is full it moves to the start of the next row; when both rows are full

it clears the screen and starts again at the top-left. It checks for a full row *before* printing, so a character is never written off the edge of the display.

Expected result: the display fills row by row with `HelloTestLoopHel` / `loTestLoopHelloT`, clears, and starts again at the top.

Task 1 exercises

1a Run it

Upload `Task1_Loopback` from your display's folder, wire the jumper and confirm the messages appear. **(record)** Take a photo of the LCD.

1b Break the loop

While the sketch is running, pull the D1-D0 jumper out, wait a few seconds, then put it back. What happens on the LCD, and why? Is anything lost while the wire is out?

Your answer:

1c Without the wrapping

In the receive loop, temporarily replace `lcdPutChar(c);` with `lcd.write(c);` and run it again. Describe what you see (compare with Figure 4), then put `lcdPutChar()` back. Why does the LCD not wrap on its own?

Your answer:

1d Timing

Each character is sent as a 10-bit frame (1 start bit, 8 data bits, 1 stop bit), and at 9600 baud each bit lasts $1/9600$ s. How long does it take to transmit the word `Hello` at 9600 baud? How many characters per second can a 9600-baud link carry at most?

Your answer:

3 Task 2: Communication between two boards

Objective: Board A (the sender) transmits a numbered message every second; Board B (the receiver) collects the characters into a complete line and shows it on its LCD and Serial Monitor.

Wiring

Label your boards **A** (sender) and **B** (receiver). The LCD connects to **Board B** only.

Board-to-board connections (the TX/RX wires cross over)

From		To	Purpose
A D11 (TX)	→	B D10 (RX)	A's data travels to B
A D10 (RX)	→	B D11 (TX)	Return path, used in the extension
A GND	→	B GND	Common reference voltage. Essential.

Task 2: two boards, TX/RX crossed over

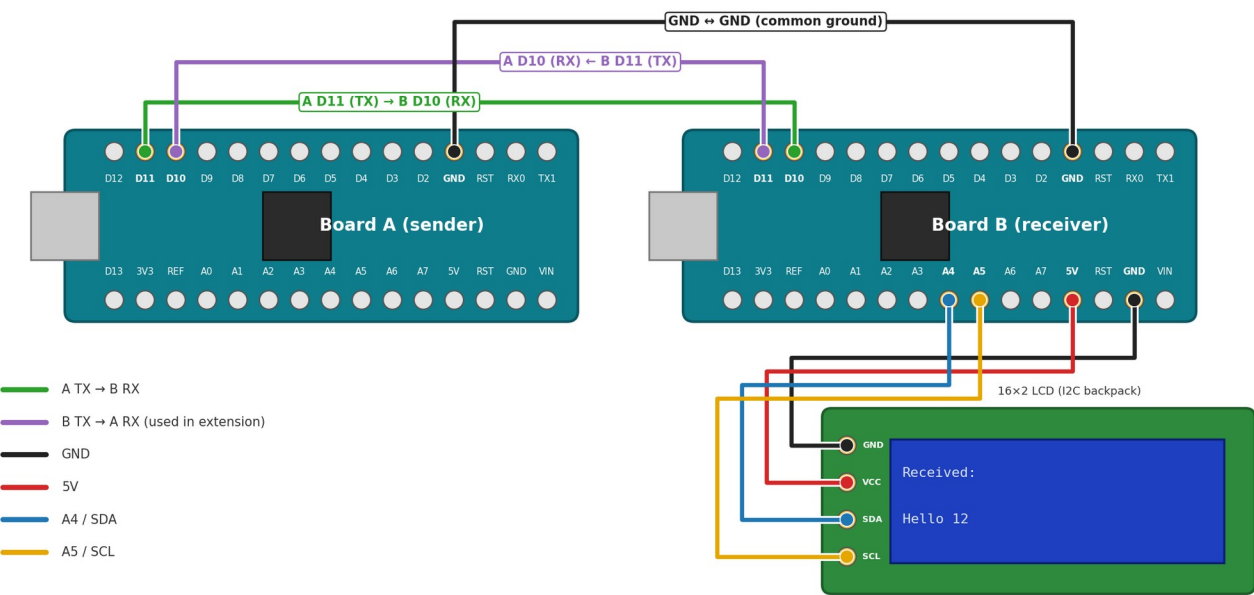


Figure 5: Task 2 wiring: TX goes to RX in both directions, grounds joined, LCD on Board B

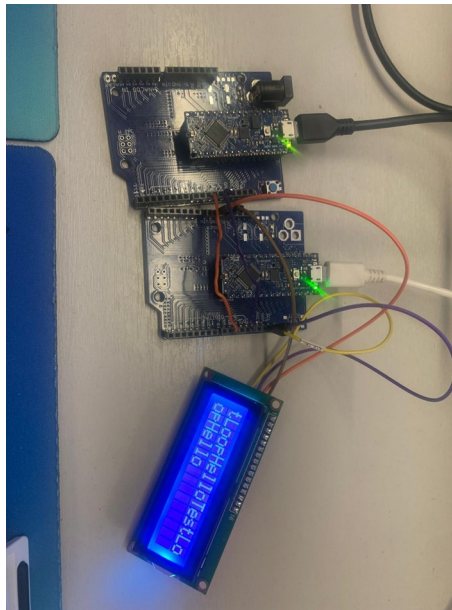


Figure 6: The two-board set-up in the lab

Connect both boards to the PC by USB. Each gets its own COM port. Upload the sender code to A and the receiver code to B, selecting the correct port each time. Open a Serial Monitor for each board, both at **9600 baud**. (In the IDE, open a second window with *File* → *New* to get a second monitor.)

Board A: sender

Sketch: Task2_Sender.ino

Board A has no LCD, so this sketch is the same in both display folders. Open it from either one:

I2C backpack LCD: `I2C_Backpack_LCD/Task2_Sender`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Task2_Sender`

- `SoftwareSerial link(10, 11)` creates a second serial port on D10 (RX) and D11 (TX). It behaves exactly like `Serial`: `link.begin()`, `link.print()`, `link.available()`, `link.read()`.
- Every second it sends one **frame**: the message text, a space, a counter and a newline, for example `Hello 12\n`. The newline marks the end of the frame, so the receiver knows where one message stops and the next begins.
- The same text goes to the USB `Serial` Monitor, so you can compare what A sent with what B received.
- Printing the frame with four separate `print()` calls is fine. At 9600 baud the wire is far slower than the CPU, so building the whole string first would not make it faster; it would only use more RAM.

The receiver problem: bytes arrive one at a time

UART delivers **individual bytes**, not whole messages. When A sends `Hello 12`, B's `read()` returns `H`, then `e`, then `l`... and there is no guarantee they all arrive in the same pass of `loop()`. The standard solution is a **line buffer**. The receiver:

1. stores each incoming character in a `char` array and increases an index;
2. when it reads `'\n'`, writes `'\0'` after the last character, turning the array into a normal C string;
3. processes the string, then resets the index to 0 ready for the next message;
4. **never** writes past the end of the array. If a message is too long, it stops storing characters, because writing past the end corrupts whatever variable sits next in memory.

Board B: receiver

Sketch: Task2_Receiver.ino

Open it from the folder for **your** display:

I2C backpack LCD: `I2C_Backpack_LCD/Task2_Receiver`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Task2_Receiver`

The two versions are the same except for the lines that set up the LCD. Read the comments in the sketch as you go: they explain every step.

- `char buffer[MAX_LEN + 1]` has one extra byte for the `'\0'` terminator. `len` counts how many characters are stored so far.
- The `while (link.available() > 0)` loop implements the four line-buffer steps above. `buffer[len] = c;` stores the character and `len++` moves the index on to the next free place.
- `else if (len < MAX_LEN)` is the overflow guard. Characters beyond `MAX_LEN` are dropped, so a long message is cut short rather than overwriting memory.
- `showMessage()` prints the completed line to the Serial Monitor. On the LCD it wipes the screen with `lcd.clear()` and prints Received: on the top row; `lcd.setCursor(0, 1)` then moves to column 0 of row 1 (rows and columns count from 0) for the message itself.

Task 2 exercises

2a Run it

Upload both sketches and run them. **(record)** Paste a few lines from **both** Serial Monitors. They should look like this:

Board A Serial Monitor

```
Sender ready
Sent: Hello 0
Sent: Test 1
Sent: Loop 2
```

Board B Serial Monitor

```
Received: Hello 0
Received: Test 1
Received: Loop 2
```

2b Break it on purpose

Make each change below **one at a time**, observe what Board B shows, then undo it. **(record)** Fill in the table and explain each result. Think about the baud rate (both boards must use the same one), which pin drives which (TX must go to RX), and the shared ground.

Change	What Board B shows	Why
Remove the GND-GND wire		
Change the receiver to <code>link.begin(4800)</code>		
Connect A's TX to B's TX instead of RX		
Set <code>MAX_LEN</code> to 4 on the receiver		

Note

A missing ground sometimes *appears* to work, because the boards may share a ground through the PC's USB ports. It is still wrong, and it fails as soon as one board runs from a battery.

2c Questions

1. Why does the receiver need the `'\n'` at the end of each frame? What would B display if the sender left it out?
2. What would happen if the `len < MAX_LEN` check were removed and A sent a 100-character message?

Your answer:

4 Task 3: Message integrity with SHA-1

Objective: noise, loose wires and baud-rate errors can flip bits in transit, and UART has no built-in error checking. In this task Board A attaches a **hash** of each message, and Board B recomputes the hash to decide whether the message arrived intact. The wiring is the same as Task 2, plus a push button on Board A that lets you simulate a transmission error.

Task 3: two boards + tamper button on Board A

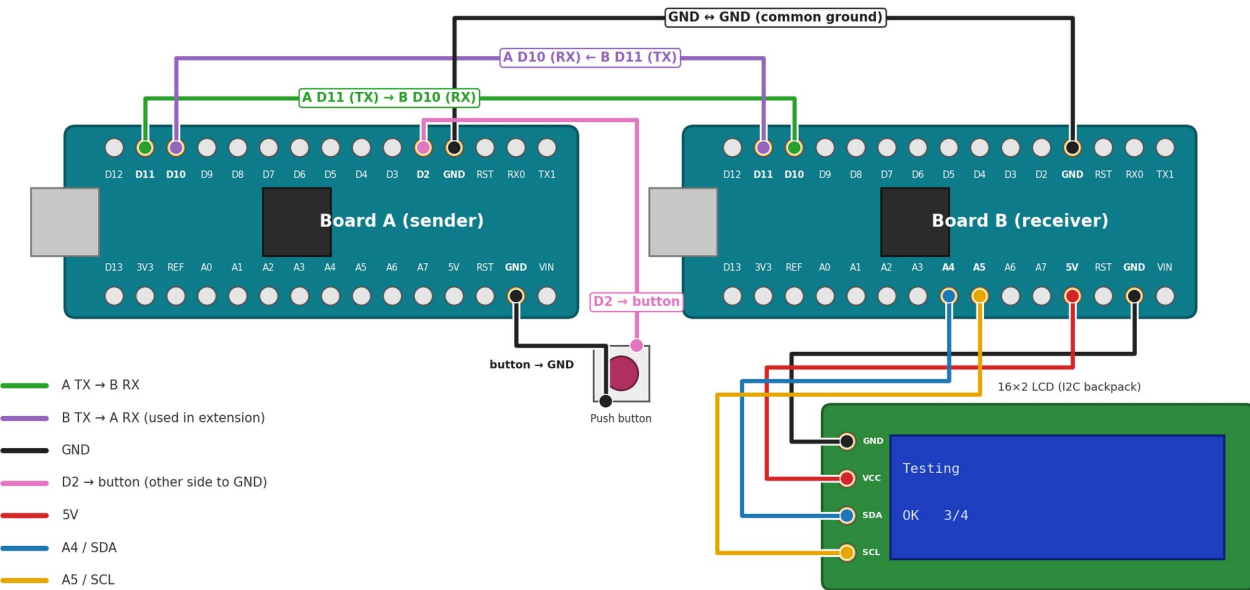


Figure 7: Task 3 wiring: the Task 2 link, plus a push button between Board A D2 and GND

Background: hash functions

A hash function turns input of any length into a **fixed-size fingerprint** called a *digest*. SHA-1 always produces 160 bits, written as **40 hexadecimal characters**. Three properties make it useful here:

- **Deterministic:** the same input always gives the same hash, so both boards will agree.
- **Avalanche effect:** changing even one bit of the input changes roughly half the output bits.
- **One-way:** you cannot work backwards from a hash to the message.

Message	SHA-1 hash
Testing	0820b32b206b7352858e8903a838ed14319acdfd
Tasting	ea1e43d415e84b553d0625a9ac75af4ea2a27fe9

One changed letter gives a completely different hash. So if the receiver's hash of the message matches the one that was sent, the message is almost certainly unchanged; if it doesn't match, something was corrupted.

Integrity is not security

SHA-1 is no longer considered secure against deliberate attacks, and a plain hash does not stop an attacker anyway: they can change the message **and** send a new matching hash. Here we only want to detect **accidental** errors. Protecting against tampering needs a secret key, for example an HMAC.

The SHA-1 function

There is no extra file to add this week. The SHA-1 code is already at the bottom of every sketch that needs it (Task 3 and the extension, both boards), under the comment SHA-1 HASH FUNCTION. You do not need to change it or understand every detail; its comments explain the three main steps (padding, mixing and output). You only need to know how to call it:

`sha1Hex(msg, hash)` works out the SHA-1 of the text `msg` and writes it into `hash` as 40 lowercase hex characters plus the `'\0'` terminator, so `hash` must be declared as `char hash[41]`. For example, the hash of Testing is 0820b32b...319acdfd.

Memory note

The textbook SHA-1 builds an 80-word table (320 bytes) for each 64-byte block. The lab's version keeps only the last 16 words (64 bytes) and computes the rest on the fly. It gives the same result but saves 256 bytes of stack, over 12% of a classic Nano's RAM.

The protocol

Each message is sent as a single line: the text, a `|` separator, the hash, and a newline. Because the frame still ends in `'\n'`, the line-buffer receiver from Task 2 works unchanged. It just needs a bigger buffer.

Task 3 frame format

Testing		0820b32b...319acdfd	\n
message (any length)	separator	SHA-1 hash: 40 hex characters	end of frame

Board A: sender with hashing and tamper button

Sketch: Task3_Sender.ino

Board A has no LCD, so this sketch is the same in both display folders. Open it from either one:

I2C backpack LCD: `I2C_Backpack_LCD/Task3_Sender`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Task3_Sender`

- `char hash[41]` holds 40 hex characters plus the `'\0'` terminator. Forgetting the `+1` is a classic buffer-overflow bug.
- The hash is always computed from the **original** message in `messages[]`, before anything can be changed.
- **Simulating an error.** `pinMode(BUTTON_PIN, INPUT_PULLUP)` makes D2 read HIGH normally and LOW while the button is pressed. The message is first copied into `msg[]` with `strncpy()`, because the original in `messages[]` is `const` and cannot be changed. While the button is held, `msg[0] ^ 0x01` flips the lowest bit of the first character, so E becomes D.
- No debouncing is needed: the button is only read once every 2 s, when a frame is sent.

- At 9600 baud a 50-byte frame takes about 52 ms, so the link is idle most of the time.

Board B: receiver with hash checking

Sketch: Task3_Receiver.ino

Open it from the folder for **your** display:

I2C backpack LCD: `I2C_Backpack_LCD/Task3_Receiver`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Task3_Receiver`

The two versions are the same except for the lines that set up the LCD. Read the comments in the sketch as you go: they explain every step.

- The receive loop is the Task 2 line buffer, with `MAX_LEN` raised to **64** (a 7-8 letter message + `|` + 40 hash characters, plus room to spare).
- **The overflow flag.** If a character arrives when the buffer is full, the flag is set, and when the `'\n'` arrives the whole frame is discarded. Without it, a too-long frame would be cut short and then fail the hash check for the wrong reason.
- `handleFrame()` uses `strchr()` to find the `|`, then splits the frame in place (see the box below). It hashes the message part with `sha1Hex()` and compares the result with the received hash using `strcmp()`, which returns 0 when two strings are equal.
- The result goes to the Serial Monitor in full and to the LCD in short form: the message on the top row, and the result with a running count, e.g. OK 3/4, on the bottom row.

How `strchr()` splitting works

Before: `T e s t i n g | 0 8 2 0 ... \0`

After `*bar = '\0'`: `T e s t i n g \0 0 8 2 0 ... \0`

No copying is needed. `frame` now reads as "Testing" and `bar + 1` reads as the 40-character hash.

Task 3 exercises

3a Run it

Upload `Task3_Sender` to Board A and `Task3_Receiver` from your display's folder to Board B, and run them. **(record)** Paste the output from Board B's Serial Monitor. Every message should pass:

Board B Serial Monitor (one frame)

```
Message: Testing
Received: 0820b32b206b7352858e8903a838ed14319acdfd
Computed: 0820b32b206b7352858e8903a838ed14319acdfd
Integrity: OK
```

3b Simulate a transmission error

Hold the button on Board A while a message is sent. **(record)** Show the FAIL result on Board B and the matching line on Board A:

Board A Serial Monitor (button held)

```
Sent (TAMPERED): Dmbedded|0fd17b94c9576216ee0a47001e3fdbb1920a1f50
```

Board B Serial Monitor (tampered frame)

```
Message: Dmbedded  
Received: 0fd17b94c9576216ee0a47001e3fdbb1920a1f50  
Computed: 106b572c38133fff8835b5165f4d092b9042db0e  
Integrity: FAIL
```

Only **one bit** of one character changed. How many of the 40 hex characters of the two hashes are different?

Your answer:

3c Questions

1. The message **Testing** is 7 bytes, but the frame is 49 bytes. What percentage of the link's capacity is spent on the hash? When would a smaller check, such as a checksum or CRC-8, be a better choice?
2. If the corruption hits the **hash** instead of the message, what does the receiver report? Is that the right outcome?
3. The receiver can detect errors but not fix them. How could the two boards use the second wire (B TX → A RX) so that a corrupted message gets sent again? (The extension below is one answer.)

Your answer:

3d Make the result stand out on your display

Do the part for **your** display. Work in a copy of **Task3_Receiver** (File → Save As...) so the original still works.

I2C backpack LCD: tick and cross. The LCD lets you design up to eight characters of your own. Each one is 5 pixels wide and 8 rows high, stored as 8 numbers (one per row), where each 1 bit is a lit pixel, e.g. 0b00011.

- Draw a tick and a cross on a 5 × 8 grid. Above `setup()`, write each one as an array of 8 numbers: `uint8_t tick[8]` and `uint8_t cross[8]`.
- In `setup()`, after `lcd.init()`, save them in the LCD with `lcd.createChar(0, tick)` and `lcd.createChar(1, cross)`, then call `lcd.clear()`.
- At the end of `handleFrame()`, move to the bottom-right corner with `lcd.setCursor(15, 1)` and show character 0 for OK or 1 for FAIL, using `lcd.write((uint8_t)0)` or `lcd.write((uint8_t)1)`.
- Question: the LCD stores each custom character as 8 bytes. Why do you think there is only room for eight of them?

Adafruit RGB LCD shield: colour-coded result. The backlight is made of a red, a green and a blue LED, and `lcd.setBacklight()` takes a 3-bit number: bit 0 is red, bit 1 is green and bit 2 is blue. The colour names (RED, GREEN, ...) are already defined near the top of the sketch.

- At the end of `handleFrame()`, call `lcd.setBacklight(GREEN)` when the hashes match and `lcd.setBacklight(RED)` when they don't.
- Run it and tap the tamper button on Board A: the screen should turn red for the corrupted frame, then green again.
- Question: which LEDs are on for YELLOW (0x3) and for WHITE (0x7)? Explain using the bits.

(record) Take a photo of your display showing an OK and a FAIL result, and answer the question for your display.

Your answer:

5 Extension: acknowledgements (ACK/NACK)

Detecting an error is only half the job: the message still needs to get through. In this extension Board B replies to every frame on the return wire (B D11 → A D10): **ACK** if the hash matched, **NACK** if it didn't. Board A resends the same message after a **NACK**, or if no reply arrives within 500 ms, and gives up after 5 retries. This is the same idea that TCP and many radio protocols use. The wiring is the same as Task 3.

Board A: sender with retries

Sketch: Extension_Sender_ACK.ino

Board A has no LCD, so this sketch is the same in both display folders. Open it from either one:

I2C backpack LCD: `I2C_Backpack_LCD/Extension_Sender_ACK`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Extension_Sender_ACK`

- **A small state machine.** The `bool` variable `waitingForReply` holds the state: `false` means idle (waiting until it is time to send the next message) and `true` means a frame is out and the sender is waiting for B's answer. `loop()` never blocks: it checks the time, reads any reply characters, and checks the timeout, then returns.
- `sendFrame()` is the Task 3 send code moved into a function, so it can be called both for a new message and for a retry. It records `sentAt` to start the timeout.
- Replies are collected with the same line-buffer pattern, into a small `reply[8]` buffer.
- `handleReply()`: `ACK` means the message was delivered, so `nextMessage()` moves on. `NACK` calls `retry()`, which resends the same message, or gives up after `MAX_RETRIES` so a permanent fault cannot lock the sender up forever.
- A missing reply is treated like a `NACK`. That covers the case where the frame, or the reply itself, was lost completely.
- Here `lastSend = millis()` is used on purpose: the next message is scheduled 2 s after the previous one *finished*, however many retries it took.

Board B: receiver with replies

Sketch: Extension_Receiver_ACK.ino

Open it from the folder for **your** display:

I2C backpack LCD: `I2C_Backpack_LCD/Extension_Receiver_ACK`

Adafruit RGB LCD shield: `Adafruit_RGB_LCD_Shield/Extension_Receiver_ACK`

The two versions are the same except for the lines that set up the LCD. Read the comments in the sketch as you go: they explain every step.

This is the Task 3 receiver with one addition: `sendReply()` sends `ACK\n` or `NACK\n` back to A. It is called for every frame, including bad or too-long frames, so the sender is never left waiting for a reply that will not come.

Why SoftwareSerial still works here

A SoftwareSerial port cannot receive while it is transmitting. That is fine here because the two boards take turns: A only listens after it has finished sending a frame, and B only replies after it has received the whole frame.

Extension exercises

E1 Run it

Upload both sketches. **(record)** Tap the button briefly so that one or two frames are corrupted, and paste Board A's output. You should see a retry succeed:

Board A Serial Monitor

```
Sent: Testing
ACK
Sent (TAMPERED): Dmbedded
NACK
Retry 1
Sent: Embedded
ACK after 1 retries
```

E2 Hold and unplug

(record) (a) Hold the button down for several seconds. What does Board A do, and why is giving up better than retrying forever? (b) Pull out the B TX → A RX wire for a few seconds. What does A report now, and what happens when you reconnect it?

Your answer:

6 Submission checklist

- Task 1: LCD photo and answers to 1b, 1c and 1d.
- Task 2: Serial Monitor output from both boards, the completed “break it” table, and answers to 2c.
- Task 3: an OK and a FAIL output from both boards, answers to 3b and 3c, and your photo and answer for 3d.
- Extension (optional): Board A output showing a successful retry, and answers to E2.